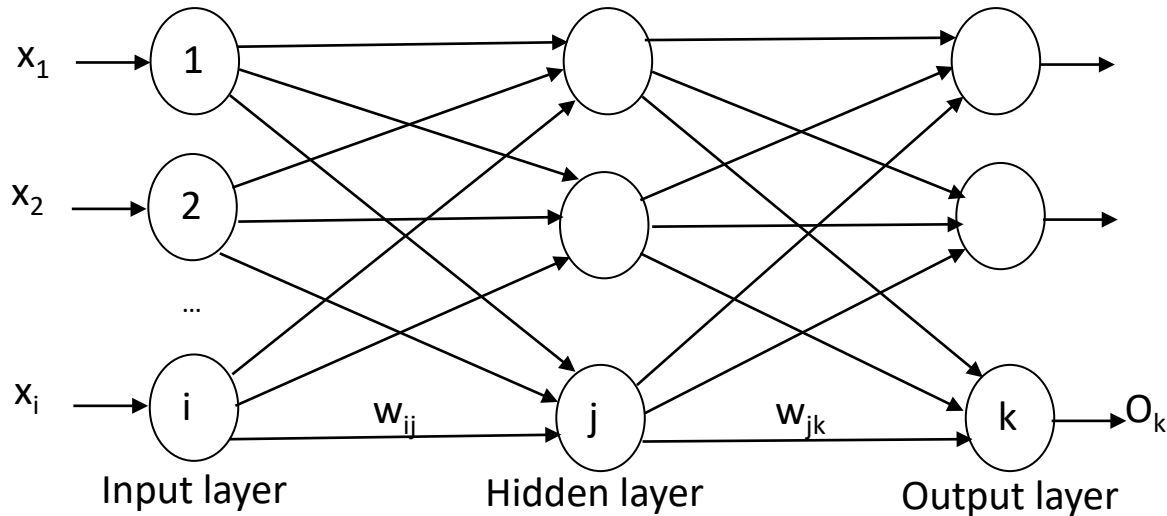


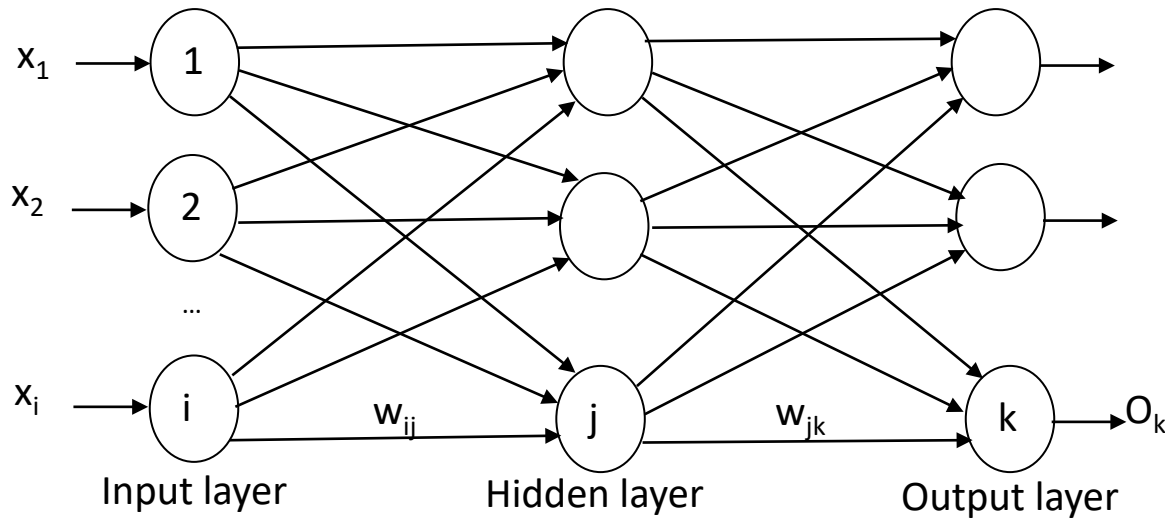
Topic 7.4 Neural Network Learning

A) An abstract example of a simple artificial Neural Network



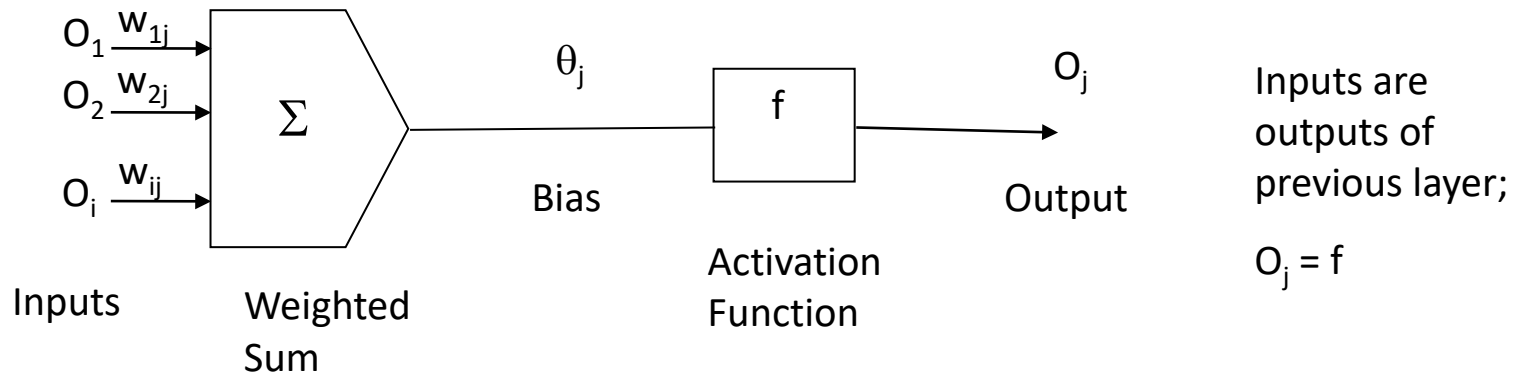
Two-layer fully connected feed-forward Neural Network

1. $(x_1, x_2, \dots, x_i)$ – numerically scaled and normalized attribute values of a sample
2. The input layer only copies
3. May be more than one hidden layers
4. Connections are weighted



5. Weighted output of one layer is passed on to the next.
6. Simply, a Neural Network is a set of connected input/ output units, where each connection has a weight associated with it.
7. A unit in hidden layer or output layer is called a neurode (neuron like) or output unit.
8. Training Samples are fed and network parameters like weights are adjusted based on feedback (the last layer output).
9. Initial topology (layers, units, connectivity, ...) and set of parameters like weights affect the performance, and can be reset if can't be trained up to the mark.
10. Various types of networks are possible: Acyclic or feed forward; Cyclic or recurrent; Single layer (hidden) / Multilayer.

B. A closer look into a neurode, j:

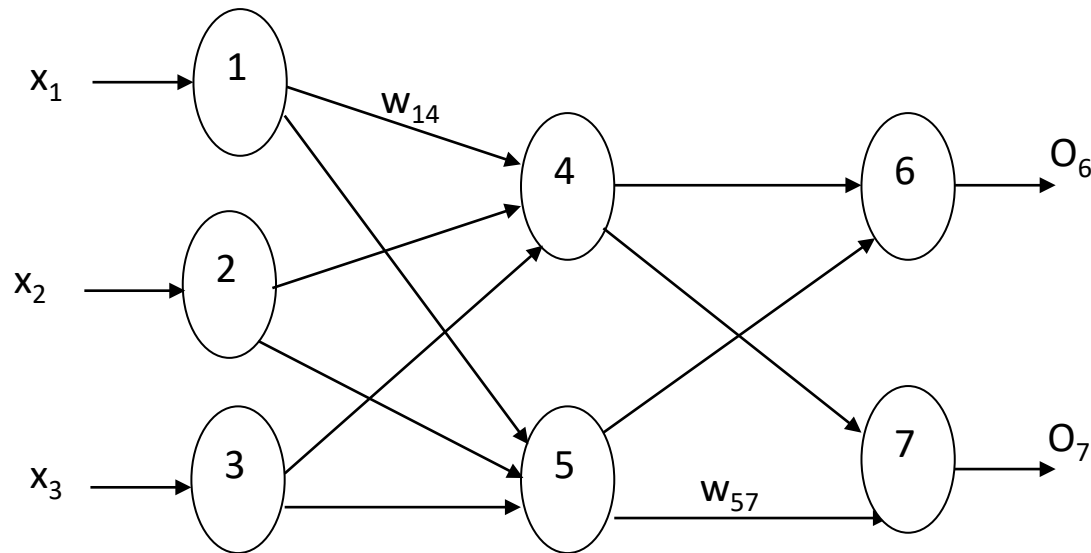


- θ_j : to vary the influence; threshold to determine whether to fire or not
- Net input, $I_j = x = \sum_i O_i * w_{ij} + \theta_j$ [O_i - output of a previous layer unit]
- $O_j = 1 / (1 + e^{-x})$
 - ✓ One type of activation functions
 - ✓ Squashing function (maps large domain to $[0, 1]$)
 - ✓ Nonlinear
 - ✓ Sigmoid function (S-shaped) / Logistic curve ('population' growth rate); Widely used in statistics

C. Features of Neural Network Learning

1. It is a kind of statistical learning methods (finding predictive function from field data).
2. Basic idea: Models information processing aspects of neurons.
3. Learning Phase: Adjusting parameters like weights and biases, so as to be able to predict correct class of an input.
4. Disadvantages: Long training time; Topology, weights, etc., are determined empirically; Poor interpretability.
5. Advantages: High tolerance to noisy data; High ability to classify patterns.
6. Backpropagation is the most popular algorithm for NNL.

D) Example of computations of NNL by Backpropagation algorithm



Input:

$$x_1 = 1$$

$$x_2 = 0$$

$$x_3 = 1$$

Biases:

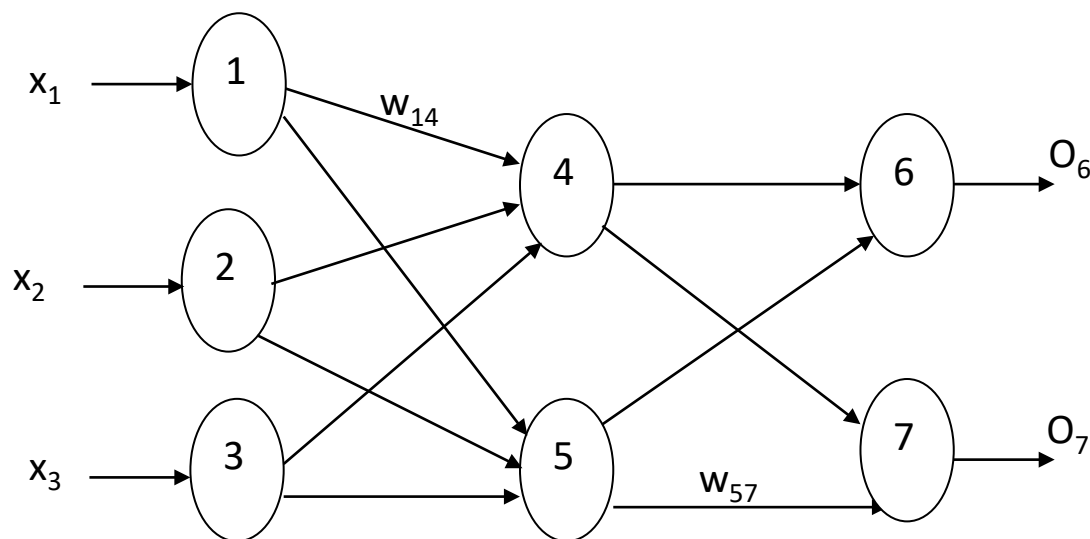
θ_4	θ_5	θ_6	θ_7
-0.4	0.2	0.1	-0.1

Weights:

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{47}	w_{56}	w_{57}
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	0.1	-0.2	0.1

i) Calculation of net input and output

Unit, j	I_j	O_j
4	$(1 \cdot 0.2) + (0 \cdot 0.4) + (1 \cdot (-0.5)) + (-0.4) = -0.7$	$1/(1 + e^{0.7}) = 0.332$
5	$(1 \cdot (-0.3)) + (0 \cdot 0.1) + (1 \cdot 0.2) + (0.2) = 0.5$	$1/(1 + e^{-0.5}) = 0.525$
6	$(0.332 \cdot (-0.3)) + (0.525 \cdot (-0.2)) + (0.1) = -0.105$	$1/(1 + e^{0.105}) = 0.474$
7	$(0.332 \cdot 0.1) + (0.525 \cdot 0.1) + (-0.1) = -0.01$	$1/(1 + e^{0.1}) = 0.496$



Input:
 $x_1 = 1$
 $x_2 = 0$
 $x_3 = 1$

Weights:

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{47}	w_{56}	w_{57}
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	0.1	-0.2	0.1

Biases:

θ_4	θ_5	θ_6	θ_7
-0.4	0.2	0.1	-0.1

ii) Error calculation

- ✓ Output layer, $\text{Err}_j = O_j (1 - O_j) (T_j - O_j)$ [$O_j(1 - O_j) = df/dl_j$ – gradient descent]
- ✓ Hidden layer, $\text{Err}_j = O_j (1 - O_j) \sum_k \text{Err}_k * w_{jk}$ [k – next layer unit]

Unit, j	Error
6	$0.474(1 - 0.474)(1 - 0.474) \approx 0.1311$
7	$0.496(1 - 0.496)(1 - 0.496) \approx 0.126$
4	$0.332 (1 - 0.332) [0.1311 * (-0.3) + 0.126 * 0.1] \approx -0.0059$
5	$0.525 (1 - 0.525) [0.1311 * (-0.2) + 0.126 * 0.1] \approx -0.0034$

True output:
 $T_6 = 1$
 $T_7 = 1$

Weights:

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{47}	w_{56}	w_{57}
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	0.1	-0.2	0.1

Biases:

θ_4	θ_5	θ_6	θ_7
-0.4	0.2	0.1	-0.1

iii) Updating Weights and Biases

■ Weights: $w_{ij} = w_{ij} + \Delta w_{ij}$
 $\Delta w_{ij} = l * Err_j * O_i$

■ Biases: $\theta_j = \theta_j + \Delta \theta_j$
 $\Delta \theta_j = l * Err_j$

Learning rate: Taken, $l = 0.9$; $[0,1]$;
 Helps avoid local maxima;
 Heuristics; May be taken, $l = 1/\text{No. of iterations}$.

$$w_{46} = -0.3 + 0.9 * 0.1311 * 0.332 \approx -0.261$$

$$w_{47} = 0.1 + 0.9 * 0.126 * 0.332 \approx 0.137$$

$$w_{56}, w_{57}, w_{14}, w_{15}, w_{24}, w_{25}, w_{34}, w_{35}$$

$$\theta_6 = 0.1 + 0.9 * 0.1311 \approx 0.218$$

$$\theta_7 = \dots\dots\dots$$

$$\theta_4, \theta_5$$

E. Features of the Backpropagation Algorithm

1. Learning is arranged by iteratively processing training samples.
2. Weights and biases are updated based on 'mean squared error' between the network prediction (output) and the actual class (set label), propagated backward from the output layer toward the input layer, layer by layer.
3. In case of 'Case updating', which is usual, updates are done after each sample input is processed.
4. In case of 'Epoch updating' accumulated error after passing all samples of the training set is propagated backward.
5. Initial weights and biases are taken randomly from a range, say, -1.0 to 1.0 or -0.5 to 0.5.
6. The process terminates if, (a) The change after an epoch is less than threshold; or, (b) A fixed number of epochs have been run; or, (c) The performance (rate of misclassification) with respect to previous epoch is below a threshold.
7. Convergence is not guaranteed, 100s of 1000s of epochs may be required, and the frustration very often leads to setting everything anew, from training samples and network topology to weights and biases.